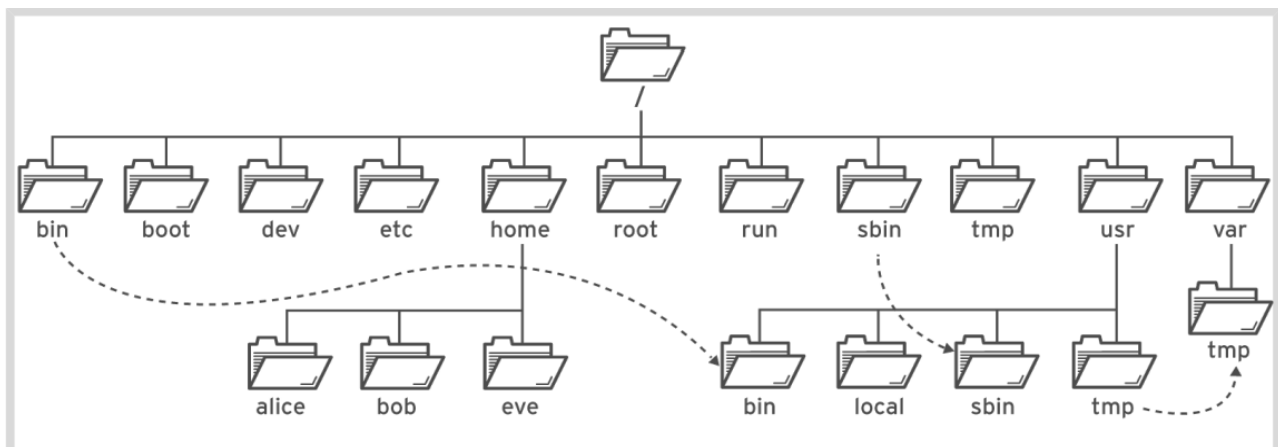


Chapter 3

Managing Files From the Command Line

➤ Describing Linux File System Hierarchy Concepts

The File-system Hierarchy



Location	Purpose
<code>/usr</code>	Installed software, shared libraries, include files, and read-only program data. Important subdirectories include: • <code>/usr/bin</code>: User commands. • <code>/usr/sbin</code>: System administration commands. • <code>/usr/local</code>: Locally customized software.
<code>/etc</code>	Configuration files specific to this system.
<code>/var</code>	Variable data specific to this system that should persist between boots. Files that dynamically change, such as databases, cache directories, log files, printer-spoiled documents, and website content may be found under <code>/var</code> .
<code>/run</code>	Runtime data for processes started since the last boot. This includes process ID files and lock files, among other things. The contents of this directory are recreated on reboot. This directory consolidates <code>/var/run</code> and <code>/var/lock</code> from earlier versions of Red Hat Enterprise Linux.
<code>/home</code>	<i>Home directories</i> are where regular users store their personal data and configuration files.
<code>/root</code>	Home directory for the administrative superuser, root .
<code>/tmp</code>	A world-writable space for temporary files. Files which have not been accessed, changed, or modified for 10 days are deleted from this directory automatically. Another temporary directory exists, <code>/var/tmp</code> , in which files that have not been accessed, changed, or modified in more than 30 days are deleted automatically.
<code>/boot</code>	Files needed in order to start the boot process.
<code>/dev</code>	Contains special <i>device files</i> that are used by the system to access hardware.

Managing Files Using Command-line Tools

Activity	Command Syntax
Create a directory	<code>mkdir <i>directory</i></code>
Copy a file	<code>cp <i>file new-file</i></code>
Copy a directory and its contents	<code>cp -r <i>directory new-directory</i></code>
Move or rename a file or directory	<code>mv <i>file new-file</i></code>
Remove a file	<code>rm <i>file</i></code>
Remove a directory containing files	<code>rm -r <i>directory</i></code>
Remove an empty directory	<code>rmdir <i>directory</i></code>

Making Links Between Files

Creating Hard Links

```
[user@host ~]$ ln newfile.txt /tmp/newfile-hlink2.txt
[user@host ~]$ ls -l newfile.txt /tmp/newfile-hlink2.txt
-rw-rw-r--. 2 user user 12 Mar 11 19:19 newfile.txt
-rw-rw-r--. 2 user user 12 Mar 11 19:19 /tmp/newfile-hlink2.txt
```

View i-node Number

```
[user@host ~]$ ls -il newfile.txt /tmp/newfile-hlink2.txt
8924107 -rw-rw-r--. 2 user user 12 Mar 11 19:19 newfile.txt
8924107 -rw-rw-r--. 2 user user 12 Mar 11 19:19 /tmp/newfile-hlink2.txt
```

Hard links have some limitations. Firstly, hard links can only be used with **regular files**. You cannot use `ln` to create a hard link to a **directory or special file**.

Creating Soft Links

The **ln -s** command creates a soft link, which is also called a "**symbolic link**." A soft link is not a regular file, but a special type of file that points to an existing file or directory.

```
[user@host ~]$ ln -s /home/user/newfile-link2.txt /tmp/newfile-symlink.txt
[user@host ~]$ ls -l newfile-link2.txt /tmp/newfile-symlink.txt
-rw-rw-r--. 1 user user 12 Mar 11 19:19 newfile-link2.txt
lrwxrwxrwx. 1 user user 11 Mar 11 20:59 /tmp/newfile-symlink.txt -> /home/user/
newfile-link2.txt
[user@host ~]$ cat /tmp/newfile-symlink.txt
Soft Hello World
```

What if We delete the Hardlink File

```
[user@host ~]$ rm -f newfile-link2.txt
[user@host ~]$ ls -l /tmp/newfile-symlink.txt
lrwxrwxrwx. 1 user user 11 Mar 11 20:59 /tmp/newfile-symlink.txt -> /home/user/
newfile-link2.txt
[user@host ~]$ cat /tmp/newfile-symlink.txt
cat: /tmp/newfile-symlink.txt: No such file or directory
```